

Revenue Cloud

Pricing

Enablement Exercises

Version 1.0, Winter '26



TABLE OF CONTENTS

Release Overview.....	3
Header Adjustments for transaction level discount.....	4
Business Objective.....	4
Use Cases.....	4
Clone Pricing Procedure and Remove Derived Pricing Elements.....	5
Configure Pricing Procedure to Use the Discount Distribution Service Element.....	6
Use the New Pricing Procedure Default.....	12
Price Revision Policy and element for CPI uplifts.....	14
Business Objective.....	14
Use Cases.....	14
Design Time Configuration.....	14
Modify the Index Rate Record Page.....	15
Create a Price Revision Policy Record.....	15
Define Index Rates- the “Price index” type of price revision policy will need to resolve the index rate to be used during transaction run time, from this object’s data.....	17
Enhancements to Formula Element.....	20
Business Objective.....	20
Use Cases.....	20
Design Time Configuration.....	20



Release Overview

Salesforce Revenue Cloud Pricing includes the following new features in the release:

1. **Header Adjustments** .
2. **Price Revision policy and pricing element for CPI uplifts**
3. **Enhanced formula element to support configuration of upto 5 formulae in a single element** - Support for adding multiple formulas in a single Formula Based Pricing element allows for a simple and a concise solution to have complex pricing calculations within the guardrail limit of allowed number of elements in the pricing procedure.
4. **Enhanced pricing performance for Large transactions:**



Header Adjustments for transaction level discount

Business Objective

Currently, Revenue Cloud lacks a straightforward mechanism to apply a discretionary discount at the transaction header level that automatically propagates to all line items. Sales representatives must manually adjust each line item, which is time-consuming and prone to error, especially on large quotes. This functionality gap complicates the sales process and is a known challenge compared to existing Salesforce CPQ capabilities.

The primary goal of the Header Adjustment - discount feature is to deliver an out-of-the-box solution that empowers sales reps to efficiently apply a single, temporary overall discount at the quote or order header. This improves sales efficiency by reducing manual calculations and simplifying the quoting process.

The key objectives are to:

1. Enable sales reps to easily apply a global discount to a transaction as an amount, percentage, or a total price override.
2. Ensure accurate, automatic distribution of the header discount to all eligible line items.
3. Provide clear visibility into how the header discount affects individual line item pricing through the Price Waterfall.
4. Seamlessly carry over applied header discounts from Quotes to Orders.

Use Cases

Sales Rep Persona:

- **Apply a Percentage Discount:** As a Sales Rep, I want to apply a 10% discount to an entire quote. I can use the "Manage Header Adjustment" action, select "Percentage," and enter "10." The system will automatically calculate and apply the 10% discount to every eligible line item.
- **Apply a Fixed Amount Discount:** As a Sales Rep, I want to apply a \$1000 total discount to an order. Using the "Manage Header Adjustment" action, I can select "Amount,"



enter "1000," and choose to distribute it either **proportionally** across line items based on their value or **equally** among all eligible lines.

- **Set a Target Total Price:** As a Sales Rep, I need to make a quote total exactly \$9,500 to win a deal. I can select "Override Total," enter "9500" as the value, and the system will calculate the necessary discount amount and distribute it across the line items to reach that target total.
- **View Discount Impact:** As a Sales Rep, I want to see how the header discount affects each line item. I can view the Price Waterfall for any line item to see a distinct entry for the "Header Discount" and understand its impact on the Net Unit Price.
- **Convert Quote to Order:** As a Sales Rep, when I convert a Quote with an applied header discount to an Order, I want the discount details (type, value, logic) to be automatically carried over and applied to the Order without extra work.

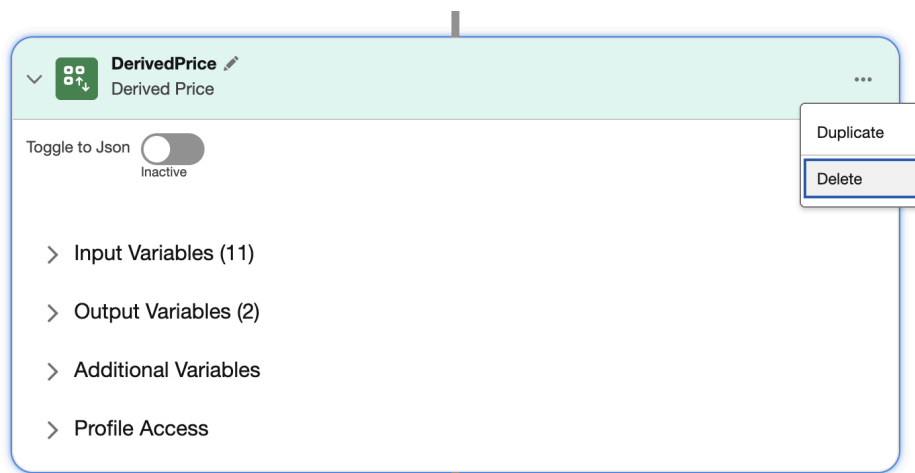
Design Time Configuration

Set Up Header Adjustments When Derived Pricing Isn't Used

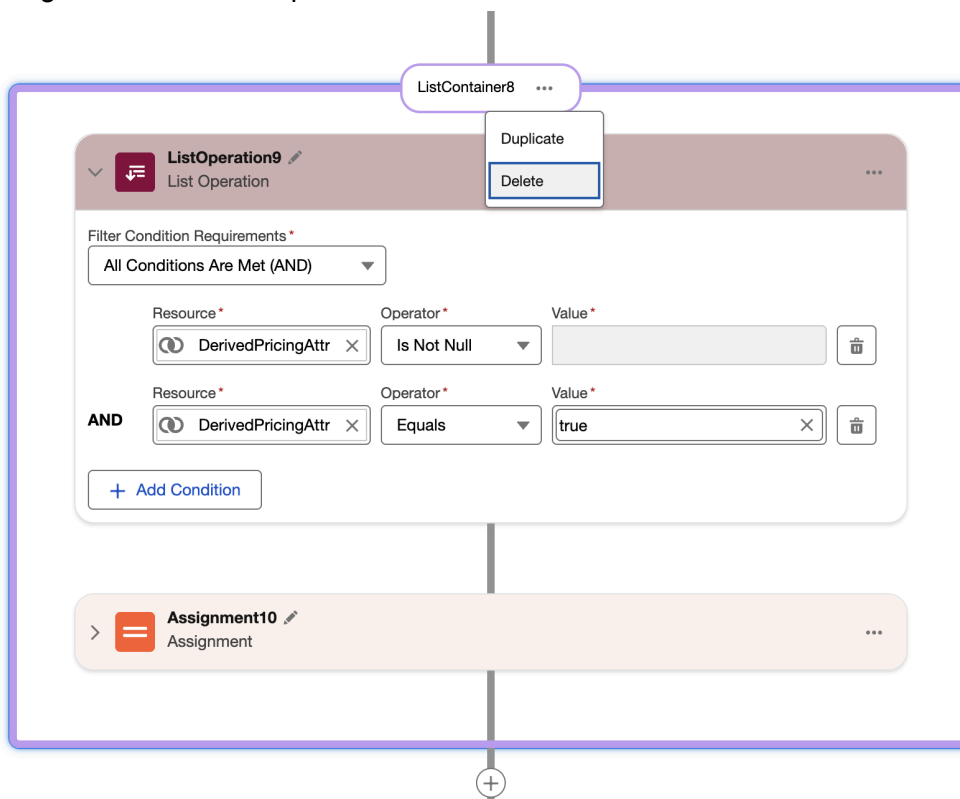
Configure your Revenue Management pricing procedure to calculate and apply header adjustments when you don't have any products that use derived pricing. Remove derived pricing elements, configure the procedure to use the Discount Distribution Service element, and set your pricing procedure as the default.

Clone Pricing Procedure and Remove Derived Pricing Elements

1. Clone the pricing procedure.
 - a. From the App Launcher, find and select **Pricing Procedures**.
 - b. Click the pricing procedure for Transaction Management.
 - c. Click **Clone**.
 - d. If needed, edit the name and the API name.
 - e. Save your changes.
2. To open your pricing procedure in Expression Set Builder, on the Details tab, in the Pricing Procedure Versions section, click the name of the pricing procedure version.
3. Delete the Derived Pricing elements.
 - a. Click ... (ALT: Settings) on the Derived Price element, and then select **Delete**.



- a. On the List Group element that contains conditions and assignments for derived pricing, click the Show Options button, and then select **Delete**.



Configure Pricing Procedure to Use the Discount Distribution Service Element

1. Add a constant that stores the header distribution type.



- a. Click  (ALT: Resource Manager).
 - b. Click **Add Resource**.
 - c. For Resource Type, select **Constant**.
 - d. For Resource Name, enter *Constant_DDS_NetUnitPrice*.
 - e. For Data Type, select **Text**.
 - f. For Value, enter *NetUnitPrice*.
2. Repeat the previous step to add these constants.
 - a. For Name, enter *Constant_DDS_Amount*; for Type, select Text; and for Value, enter Amount.
 - b. For Name, enter *Constant_DDS_Percentage*; for Type, enter Text; and for Value, enter Percentage.
 - c. For Name, enter *Constant_DDS_Override*; for Type, select Text; and for Value, enter Override.
3. Add a List Group and configure it to assign values when the discount type is Amount.
 - a. After the last component in the pricing procedure, click  (ALT: Add component), and then click **List Group**. A group is added along with the List Operation element.
 - b. From Filter Condition Requirements, select **All Conditions Are Met (AND)**.
 - c. In the Resource field, select **AppliedDiscountAmount__std**, and then in the Operator field, select **Is Not Null**.
 - d. Click **Add Condition**.
 - e. In the Resource field, select **AppliedDiscount__std**, and then in the Operator field, select **Is Null**.
 - f. Click **Add Condition**.
 - g. In the Resource field, select **TotalPriceOverride__std**, and then in the Operator field, select **Is Null**.
 - h. After the List Operation component in the container, click  (ALT: Add component), and then click **Assignment**.
 - i. For Input Variable, select **AppliedDiscountAmount__std**, and for Output Variable, select **HeaderAdjustmentValue__std**.
 - j. Click **Add more variables**.



- k. For Input Variable, select **Constant_DDS_Amount**, and for Output Variable, select **HeaderAdjustmentType__std**.



List Container ...

▼

List Operation

List Operation

...

Filter Condition Requirements *

All Conditions Are Met (AND) ▼

Resource *

AppliedDiscountA

Operator *

Is Not Null ▼

Value *

AND

Resource *

AppliedDiscount_

Operator *

Is Null ▼

Value *

AND

Resource *

TotalPriceOverride

Operator *

Is Null ▼

Value *

+ Add Condition

▼

Assignment

Assignment

...

Toggle to Json Inactive

▼ Assignment variables

Input Variable *

AppliedDiscountAmount__std

Output Variable *

HeaderAdjustmentValue__std

Input Variable *

Constant_DDS_Amount

Output Variable *

HeaderAdjustmentType__std

+ Add more variables

> Profile Access



4. Add a List Group that assigns values when the discount type is Percentage.

- a. After the last component in the pricing procedure, click  (ALT: Add component).
 - b. Click **List Group**. A group is added along with the List Operation element.
 - c. From Filter Condition Requirements, select **All Conditions Are Met (AND)**.
 - d. In the Resource field, select **AppliedDiscount__std**, and then in the Operator field, select **Is Not Null**.
 - e. Click **Add Condition**.
 - f. In the Resource field, select **AppliedDiscountAmount__std**, and then in the Operator field, select **Is Null**.
 - g. Click **Add Condition**.
 - h. In the Resource field, select **TotalPriceOverride__std**, and then in the Operator field, select **Is Null**.
 - i. After the List Operation component in the container, click  (ALT: Add component), and then click **Assignment**.
 - j. For Input Variable, select **AppliedDiscount__std**, and for Output Variable, select **HeaderAdjustmentValue__std**.
 - k. Click **Add more variables**.
 - l. For Input Variable, select **Constant_DDS_Percentage**, and for Output Variable, select **HeaderAdjustmentType__std**.
5. Add a List Group that assigns values to variables when the discount type is Override Total.

- a. After the last component in the pricing procedure, click  (ALT: Add component).
- b. Click **List Group**. A group is added along with the List Operation element.
- c. From Filter Condition Requirements, select **All Conditions Are Met (AND)**.
- d. In the Resource field, select **TotalPriceOverride__std**, and then in the Operator field, select **Is Not Null**.
- e. Click **Add Condition**.
- f. In the Resource field, select **AppliedDiscountAmount__std**, and then in the Operator field, select **Is Null**.
- g. Click **Add Condition**.
- h. In the Resource field, select **AppliedDiscount__std**, and then in the Operator field, select **Is Null**.
- i. After the List Operation component in the container, click  (ALT: Add component), and then click **Assignment**.



- j. For Input Variable, select **TotalPriceOverride__std**, and for Output Variable, select **HeaderAdjustmentValue__std**.
 - k. Click **Add more variables**.
 - l. For Input Variable, select **Constant_DDS_Override**, and for Output Variable, select **HeaderAdjustmentType__std**.
6. Add and configure the Discount Distribution Service element.
- a. After the last component in the pricing procedure, click  (ALT: Add component), and then select Discount Distribution Service.
 - b. Map these input variables.
 - i. Header Discount Type: *HeaderAdjustmentType__std*
 - ii. Header Discount Value: *HeaderAdjustmentValue__std*
 - iii. Header Distribution Logic: *AdjustmentDistributionLogic__std*
 - iv. Header Distribution Type: *Constant_DDS_NetUnitPrice*
 - v. List Price: *NetUnitPrice*
 - vi. Quantity: *LineItemQuantity*
 - vii. Line Item: *LineItem*
 - viii. Input Unit Price: *NetUnitPrice*
 - ix. Header Subtotal: *TotalAmount*
 - x. Minimum Net Unit Price: *MinimumNetUnitPrice*
 - c. Map these output variables.
 - i. Discount Value: *LineItemDiscountValue*
 - ii. Distribution Type: *HeaderDistributionType*
 - iii. Discount Type: *LineItemDiscountType*
 - iv. Net Unit Price: *NetUnitPrice*



v. Subtotal: *ItemNetTotalPrice*

Input Variables (10)		
Header Discount Type *	→	HeaderAdjustmentType_std
Header Discount Value *	→	HeaderAdjustmentValue_std
Header Distribution Logic *	→	AdjustmentDistributionLogic_std
Header Distribution Type *	→	Constant_DDS_NetUnitPrice
List Price *	→	NetUnitPrice
Quantity *	→	LineItemQuantity
Line Item *	→	LineItem
Input Unit Price	→	NetUnitPrice
Header Subtotal	→	TotalAmount
Minimum Net Unit Price	→	MinimumNetUnitPrice
Output Variables (5)		
Discount Value *	→	LineItemDiscountValue
Distribution Type *	→	HeaderDistributionType
Discount Type *	→	LineItemDiscountType
Net Unit Price	→	NetUnitPrice
Subtotal	→	ItemNetTotalPrice

- vi. Save your procedure.
- vii. [Simulate and activate your pricing procedure.](#)

Use the New Pricing Procedure Default

- [Set the pricing procedure you created as the default pricing procedure.](#)

If necessary, you can also set and use floor price limits. See [Discount Distribution Service](#).





Price Revision Policy and element for CPI uplifts

Business Objective

Ensure pricing strategies are in line with the company's margin goals by linking price revisions to price indices. Some businesses are more sensitive to price index changes and want to ensure they are pricing their products / services appropriately by enforcing price revisions whenever underlying price factors change

Use Cases

Companies want to automatically apply a price uplift during renewals , that takes into account the relevant price index. There is also a need for flexibility to this uplift where a seller can negotiate further on the final uplift that should take effect.

Design Time Configuration

1. This feature includes the objects
 - a. Price Revision policies - This object captures the formula for the price revision and whether the revision is a flat rate or price index driven
 - b. Index rate- This is an OOB object for customers to load their price index data into , so it can be referenced in the formula to calculate the revised price
 - c. Index rate Decision table is also shipped OOB to be used with the price revision pricing element

Let's consider a scenario where a business wants to adapt its pricing to frequent inflationary effects in a country that we'll call ABC. To achieve this, it secures an agreement with its customers for an annual price increase equivalent to the Consumer Price Index (CPI) plus 5%, while ensuring the total adjustment is capped at 10%.

To calculate the revised price, you'll need to do the following:

1. Add picklist values to the Region field on the Index Rate entity.
2. Create a price revision policy record
3. Define the index rates
4. Add the Price Revision element to a pricing procedure to calculate the adjusted prices due to inflation



Modify the Index Rate Record Page

Before you create an index rate record, you'll need to add Region picklist values to the Index Rate entity's Region field.

1. From Setup, in Object Manager, search for and select Index Rate.
2. Select Fields & Relationships, and then click Region.
3. In the Region Picklist Values section, click New.
4. Enter country names. For our use case, specify US , Argentina as shown below
5. Save your changes.

Index Rate Field
Region
[Back to Index Rate Fields](#) [Help for this Page](#)

[Edit](#) [Set Field-Level Security](#) [View Field Accessibility](#)

Field Information

Field Label	Region	Field Name	Region
Data Type	Picklist		
Help Text			
Description			
Data Owner			
Field Usage			
Data Sensitivity Level			
Compliance Categorization			

Picklist Values Used
Active and inactive picklist values 2 (1,000 max)

Field Dependencies [New](#) [Field Dependencies Help](#)

No dependencies defined.

Validation Rules [New](#) [Validation Rules Help](#)

No validation rules defined.

Region Picklist Values [New](#) [Reorder](#) [Replace](#) [Printable View](#) [Chart Colors](#) [Region Picklist Values Help](#)

Action	Values	API Name	Default	Chart Colors	Modified By
Edit Del Deactivate	US	US	☐	Assigned dynamically	Admin User , 9/24/2025, 5:02 AM
Edit Del Deactivate	Argentina	Argentina	☐	Assigned dynamically	Admin User , 9/24/2025, 5:02 AM

Create a Price Revision Policy Record

Configure an inflation policy for the country, ABC. This policy will enable sales reps to apply specific price uplifts to quotes and deals based on the defined inflation rate in the country, ABC.

Before creating a price revision policy, you need to understand the two policy types:

- Flat: This policy type uses a manually entered uplift value (a policy adjustment). The system doesn't reference a decision table to find the value.
- Price Index: This policy type automatically retrieves the uplift value from the Index Rate decision table, ensuring the rate matches the input criteria.



1. From App Launcher, find and select Price Revision Policies.
2. Click New.
3. Specify these details.
 - a. Name: ABC CPI
 - b. Type: Price Index
 - c. Effective From: 1/1/2025
 - d. Effective To: 12/31/2025
 - e. Region: US
 - f. Formula: MAX(10, PriceIndex + 5)
To define a formula, follow the steps below.
 - i. Select a function in the Search functions... field. Here, select MAX.
 - ii. Highlight the value within the brackets and delete it.
 - iii. Place your cursor in the brackets and then, enter 10,.
 - iv. Select a resource from the Select a resource... field. Here, select PriceIndex.
 - v. Select an operator from the Enter an operator... field. Here, select +.
 - vi. Enter 5. All numerical values are treated as a percentage
 - vii. Effective date ranges for the same region cannot overlap, unless you're on a multi-currency organization.

* Formula

iv Select a resource... i Search functions... v Enter an operator...

MAX(10, PriceIndex+ 5) iii

Cancel Save

4. Save your changes



Define Index Rates- the “Price index” type of price revision policy will need to resolve the index rate to be used during transaction run time, from this object’s data

The index rates entity stores inflation rates for various periods of time based on different time periods.

1. From App Launcher, find and select Index Rates.
2. Click New.
3. Specify these details.
 - a. Name: ABC CPI 2025
 - b. Rate: 8
 - c. Valid Start Date: 1/1/2025
 - d. Valid End Date: 12/31/2025
 - e. Region: ABC
 - f. Usage Type: Pricing
4. Effective date ranges for the same region can’t overlap, unless you’re on a multi-currency organization
5. Save your changes.

Create a Pricing procedure to test the Price revision element

1. Create a pricing procedure. To create a pricing procedure, follow the first 5 steps in [Configure Your Pricing Procedure](#).
2. Add the Pricing Setting element and map these variables.
 - a. Input Variables
 - i. Line Item: **LineItem**
 - b. Output Variables
 - i. Price Waterfall: **price_water_fall**
 - ii. Net Unit Price: **NetUnitPrice**
 - iii. Subtotal: **ItemNetTotalPrice**
3. Add the List Price element to fetch the base price of the product.

Under Lookup Table Details, select the **Price Book Entries V2** decision table and map these variables.

 - a. Input Rule Variables
 - i. Product: **Product**
 - ii. Price Book: **PriceBooks**
 - iii. Product Selling Model: **ProductSellingModel**



- b. Input Variables
 - i. Quantity: **LineItemQuantity**
 - c. Output Variables
 - i. List Price: **ListPrice**
 - ii. Subtotal: **ItemNetTotalPrice**
4. Add the Price Revision element.
- Under Lookup Table Details, select the **Index Rate** decision table and map these variables.
- a. Input Rule Variables
 - i. Valid Start Date: **ItemConsumerPriceIndexDate_std**
This is the date for your price revision which must fall within the effective date range specified in the index rate record.
 - ii. Region: **ItemPriceRevisionPolicyRegion_std**
This is the region that you're verifying your inflation for.
 - b. Input Variables
 - i. Quantity: **LineItemQuantity**
 - ii. Price Revision Policy: **ItemPriceRevisionPolicyName_std**
This is the name of the price revision policy record.
 - iii. Input Unit Price: **ListPrice**
 - iv. Policy Type: **ItemPriceRevisionPolicyType_std**
This is the policy type you've selected in the price revision policy record. It can be Flat or Price Index.
 - v. Price Revision Formula: **ItemPriceRevisionPolicyFormula_std**
This is the formula you've provided in the price revision policy record.
2. Click  and select **Include in Output**.
3. Save your procedure.
4. Click **Simulate** to test your procedure.
5. Specify the input values for subscriptions to Slack along with the additional uplift of 5% that must be added to maintain gross margins. Click **Simulate** again.
- The price waterfall shows the Price Revision updates, confirming that your procedure is working as expected.



▼ Slack 10 units		
Price Details	Description	Price
Price Book Entries	Price Book Entries	\$ 99.99
Price Revision	Policy Name = ABC CPI 2025 Formula = MAX(10, PriceIndex + 5) Policy Adjustment = 5% Price Revision Value = 10% Final price per unit	\$ 109.989
Net Amount	Per unit price x 10 units	\$ 1099.89

When you simulate a pricing procedure, you can enter the formula manually to verify if the adjusted prices are reflected for a given product. However, when a sales rep creates a quote, they'll see this information on the quote itself, along with the option to enter additional uplift data.

If you see inaccurate results, please try refreshing the Index rate Decision table .



Enhancements to Formula Element

Business Objective

Formula-based pricing offers a dynamic approach to price calculation, moving beyond static price lists and discounts. This feature allows businesses to define sophisticated pricing rules using custom formulas and functions within pricing procedures.

Support for adding multiple formulas in a single Formula Based Pricing element allows for a simple and a concise solution to have complex pricing calculations within the guardrail limit of allowed number of elements in the pricing procedure.

Use Cases

Example Use Cases

As a Pricing Administrator I want to be able to add multiple sequential formula based calculations in the pricing procedure, without breaching the guardrail limit of allowed number of elements in the procedure.

Design Time Configuration

1. Open a new pricing procedure and add the price setting element and the formula element.
2. Map the context tags and add the multiple formulas in the formula based pricing element.
 - a. Click on “Add more variables and rules” to add a new formula to the element. Up to 5 formulas can be added to the element.
 - b. Evaluation of formulas will happen in the sequential order in which formulas have been added to the element.
 - c. If the output context tag of a formula is an input to the next formula, after evaluation of the first formula the updated value of the context tag will be passed to the next formula.



Pricing Setting

Pricing Setting

Toggle to Json

Inactive

Input Variables (3)

Line Item *

→

LinItem

×

Is Derived

→

Select a variable of type boolean.

Contract Id

→

Select a variable of type text.

Output Variables (3)

Price Waterfall

→

Select a variable of type text.

Net Unit Price *

→

ListPrice

×

Subtotal *

→

Subtotal

×

Formula Based Pricing

Formula Based Pricing

Toggle to Json

Inactive

Exclude Price Waterfall

Formula Variables (4)

Formula *

NetUnitPrice + 5

→

Output Variable *

NetUnitPrice

×

Formula *

ItemNetTotalPrice * 10

→

Output Variable *

ItemNetTotalPrice

×

Formula *

MAX (ItemNetTotalPrice , 1000)

→

Output Variable *

Subtotal

×

Formula *

NetUnitPrice * 7

→

Output Variable *

NetUnitPrice

×

Add more variables and rules

Additional Variables

Profile Access



Simulation

1. To illustrate calculation process, simulate the pricing procedure created above with below values:
 - a. LineItem: Laptop
 - b. NetUnitPrice: 100
 - c. ItemNetTotalPrice: 200

▼ Laptop	unit	
Price Details	Description	Price
Formula Based Pricing	NetUnitPrice + 5	\$ 105
	ItemNetTotalPrice * 10	\$ 2000
	MAX (ItemNetTotalPrice , 1000)	\$ 2000
	NetUnitPrice * 7	\$ 735

2. Explanation: -
 - a. The first formula updates the value of NetUnitPrice to \$105.
 - b. The second formula updates the value of ItemNetTotalPrice to \$2000.
 - c. The third formula takes the updated value of ItemNetTotalPrice as input, and therefore returns the output as \$2000.
 - d. The fourth formula takes the updated value of NetUnitPrice as input, and therefore returns the output as \$735.

Sample output

```
{
  "additionalOutputData": {
    "pricingWaterfall": [
      {
        "apiEndpoint": null,
        "apiExecutionId": null,
        "contextDefinitionVersionId": "11OWs000000KKPEMA4",
        "contextMappingId": "11jWs00007dcqZvIAI",
        "currencyCode": "USD",
        "error": null,
        "executionEndTimestamp": "2025-09-10T17:55:41.126Z",
        "executionId": "218892340874914",
        "executionStartTimestamp": "2025-09-10T17:55:40.589Z",
```



```
"id": "Laptop",
"lineItemId": "Laptop",
"output": {
  "isParallelExecution": false,
  "formula-section-3-output": 735,
  "formula-section-2-output": 2000,
  "formula-section-0-output": 105,
  "formula-section-1-output": 2000
},
"referenceKey": null,
"success": true,
"usageType": "Pricing",
"waterfall": [
  {
    "fieldToTagNameMapping": {
      "ItemNetTotalPrice": "ItemNetTotalPrice",
      "NetUnitPrice": "NetUnitPrice",
      "Subtotal": "Subtotal",
      "LineItemId": "LineItem",
      "formula-section-3-output": "NetUnitPrice",
      "formula-section-2-output": "Subtotal",
      "formula-section-0-output": "NetUnitPrice",
      "formula-section-1-output": "ItemNetTotalPrice"
    },
    "hideWaterfall": false,
    "inputParameters": {
      "NetUnitPrice": 100,
      "ItemNetTotalPrice": 200,
      "formula-section-3-input": "NetUnitPrice * 7",
      "formula-section-0-input": "NetUnitPrice + 5",
      "Subtotal": 4,
      "formula-section-2-input": "MIN ( ItemNetTotalPrice , 2001 )",
      "LineItemId": "Laptop",
      "formula-section-1-input": "ItemNetTotalPrice * 10"
    },
    "outputParameters": {
      "formula-section-3-output": 735,
      "formula-section-2-output": 2000,
      "formula-section-0-output": 105,
```



```
        "formula-section-1-output": 2000
      },
      "pricingElement": {
        "adjustments": [],
        "cumulativePricing": null,
        "description": null,
        "elementType": "FormulaBasedPricing",
        "name": "Formula Based Pricing"
      },
      "profileAccess": [],
      "sequence": 0,
      "tasksInfo": []
    }
  ]
}
],
},
"executionId": "9QMWs000000bAgLOAU_20250910_175541_3ccf43db",
"output": {
  "aggregationResults": {},
  "calculationResults": [
    {}
  ]
},
"simulationResults": [
  {
    "contextValue": {
      "SalesTransaction": [
        {
          "Subtotal": 2000,
          "SalesTransactionItem": [
            {
              "NetUnitPrice": 735,
              "ItemNetTotalPrice": 2000,
              "LineItem": "Laptop",
              "ListPrice": null
            }
          ]
        }
      ]
    }
  }
]
```




```
    ]
  },
  "datatype": "Context",
  "name": "PricingTransactionCD",
  "precision": 2,
  "value": ""
}
],
"simulationStepResults": {
  "PricingSetting": {
    "isDefaulted": false,
    "stepErrors": {},
    "stepInputs": [],
    "stepResults": []
  },
  "FormulaBasedPricing": {
    "isDefaulted": false,
    "stepErrors": {},
    "stepInputs": [],
    "stepResults": []
  }
}
}
```

